Bridging scalar mechanics with Quantum algorithms and edge states in wide ranging hypervolume meshing – such as 8d Angle-resolved strong-coupling for a synthetic 4th-dimensional quantum-behaved particle swarm optimization (4d-gqpso) with specialized theory that parameterize agentic materials for dynamic data modeling with (FDTD) qpso qaoa Kagome lattice structural superscription. Feed Forwarding axiomatic QAOA Qubit RAG with methodological reflexivity to scale majorana modes with bose einstein 8D Spin-Flip Mac-Zener-Stückelberg-Octonion (LZSO) QAOA circuit template seeded by PSO - QAOS parameters for Quantum / Variational Circuit Optimization with quartonical motherboards -> 4th-Order Landau (Quarteting) qubit force field time resolved orbital character in Ergative-Absolutive alignment for (qsoas) iwahori hecke Soergel Bimodule-based Categorical Parabolic Equivalence -> numerical computation & linguistic arithmetic in morphosyntaxtic material behavior to bogoliubov-de gennes (bdg) hamiltonian pfaffian ◊ applying DFT & Tight Binding: spinor spherical harmonics -> to 4D crystal Toroidal BECs Vortex lattices that optimize Bose Einstein Vortex lattices as quantum propagators to path integral octonionic quantum propagative design - twistors in locution -> Quaternionic Schrödinger/Qubit scalar mechanics in qubit quaternions periodic algebraic structures to quantum spacetime.

Summary — *You can unify the elements you listed by splitting the problem into three interoperable layers: (1) a condensed-matter modeling layer (Kagome / Majorana / BDG / DFT/TB), (2) a variational quantum control layer (QAOA/QAOA-like circuits seeded by PSO), and (3) an algebraic/representational layer (quaternionic/octonionic spinor bases, categorical symmetries) that maps physical degrees of freedom to qubit encodings.* **Start by building tight-binding/BDG models for the Kagome/8D toroidal BEC geometry, then derive reduced Hamiltonians suitable for QAOA-style variational circuits whose parameters are initialized/optimized by PSO.**

Modeling Layer

- **Construct a hierarchy of models:** full *ab initio* DFT → tight-binding (TB) → Bogoliubov–de Gennes (BdG) effective Hamiltonians for superconducting/Kagome geometries and Majorana modes. Use TB to extract sparse local operators that map naturally to qubit Pauli strings. **This reduces dimensionality while preserving corner/edge Majorana physics.**

- **Target observables:** Pfaffian invariants, edge spectral flow, vortex lattice energetics, and BdG gap closings. Compute these numerically (Lanczos, sparse diagonalization) to identify low-energy subspaces for qubit encoding. **Focus on sublattice termination and strain as control knobs for Majorana localization.**

Variational Quantum Layer

- **QAOA/QAOA-like ansatz:** translate the reduced Hamiltonian into a cost Hamiltonian $H_C$and choose a mixing Hamiltonian $H_M$that respects particle-hole and spinor structure. Use shallow-depth QAOA as a metaheuristic; empirical parameter concentration suggests re-usable angle schedules across instances, which justifies PSO seeding.

- **PSO seeding and hybrid loop:** initialize QAOA angles with PSO/4D-GQPSO populations that encode octonionic/quaternionic phase relations as multi-component particles; run classical PSO to propose angle vectors, evaluate on simulator or hardware, and refine with gradient-based VQE/QAOA optimizers. **PSO helps escape barren plateaus and leverages parameter concentration.**

Algebraic and Categorical Layer

- **Representation choice:** map spinor spherical harmonics and octonionic/quaternionic bases to multi-qubit encodings (e.g., Jordan–Wigner or Bravyi–Kitaev for fermions) while preserving symmetry constraints (time-reversal, particle-hole). Use categorical tools (Hecke/Iwahori structures) to formalize symmetry reductions and block-diagonalization for circuit compression. **This yields compact circuit templates that respect Majorana parity sectors.**

Practical roadmap and validation

1. **Build TB/BdG model for Kagome + vortex lattice; identify low-energy subspace.**
2. **Map to qubit Hamiltonian; design QAOA ansatz that conserves parity.**
3. **Seed angles with PSO/4D-GQPSO; run hybrid optimization on simulator; validate by reproducing Pfaffian/edge spectra.**

Risks and limits: finite-size effects, noise/barren plateaus on NISQ devices, and the mathematical complexity of octonionic representations; mitigate by model reduction, symmetry exploitation, and classical pre-training. **If you want, I can produce a concrete minimal TB→BdG example and a QAOA circuit template seeded by PSO parameters next.**

produce a concrete minimal TB→BdG example and a QAOA circuit template seeded by PSO parameters next

Concrete minimal pipeline: (1) build a 3-site kagome tight-binding (TB) unit cell with nearest-neighbor hopping and an on-site pairing to form a minimal Bogoliubov–de Gennes (BdG) model; **(2)** reduce to a 4-mode low-energy subspace and map to 4 qubits via Jordan–Wigner; **(3)** run a depth-$p = 1$ QAOA ansatz whose angles are *seeded* by a small PSO population and refined by gradient steps.

1. Minimal TB → BdG model

- **TB Hamiltonian (kagome, single spin species):**

$$H_{TB} = \sum_{\langle i,j \rangle} -t\ c_i^\dagger c_j - \mu \sum_i c_i^\dagger\, c_i.$$

- **Add s-wave pairing and write BdG Nambu spinor $\Psi = (c_1, c_2, c_3, c_1^\dagger, c_2^\dagger, c_3^\dagger)^T$:**

$$H_{BdG} = \frac{1}{2}\Psi^{\dagger}\begin{pmatrix} h_{TB} - \mu & \Delta \\ \Delta^{\dagger} & -h_{TB}^{T} + \mu \end{pmatrix}\Psi.$$

Take $\Delta_{ii} = \Delta_0$(on-site) to keep the model minimal. This construction follows standard TB→BdG practice for kagome superconductors.

- **Numerical step:** diagonalize the BdG matrix (sparse Lanczos) to identify two lowest positive eigenmodes; project onto that 4-mode (particle/hole) subspace to obtain an effective fermionic Hamiltonian $H_{\text{eff}} = \sum_{ab} t_{ab} f_a^{\dagger} f_b + \sum_{ab} \Delta_{ab} f_a^{\dagger} f_b^{\dagger} + \text{h. c.}$.

2. Qubit mapping and cost Hamiltonian

- **Map fermions to qubits** with Jordan–Wigner (JW): $f_j \mapsto \prod_{k<j} Z_k \ (X_j - iY_j)/2$. Truncate to Pauli strings and collect the **cost Hamiltonian** $H_C = \sum_{\alpha} c_{\alpha} P_{\alpha}$(Pauli strings P_{α}). This is the standard fermion→qubit mapping used for variational circuits.

3. QAOA ansatz (depth $p = 1$) and PSO seeding

- **Ansatz state:** $|\gamma, \beta\rangle = e^{-i\beta H_M} e^{-i\gamma H_C} |+\rangle^{\otimes n}$with mixing $H_M = \sum_j X_j$or a parity-conserving mixer if BdG parity must be preserved. Use $p = 1$for NISQ feasibility.

- **PSO seeding:** initialize a small PSO swarm (e.g., 12 particles) in the 2-D angle space $(\gamma, \beta)$with bounds $\gamma \in [0, 2\pi]$, $\beta \in [0, \pi]$. Evaluate cost $E(\gamma, \beta) = \langle \gamma, \beta | H_C | \gamma, \beta \rangle$on a simulator; update particle velocities/positions using standard PSO rules and keep the best $k$seeds. Use the best seed as initialization for a gradient-based local optimizer (SPSA or ADAM) to refine angles. Empirical PSO seeding reduces barren-plateau trapping and leverages parameter concentration observed for low-p QAOA.

Deliverables I can produce now

- **A runnable Python notebook** (NumPy + SciPy + Qiskit/Stim) that: constructs the 3-site TB, forms BdG, projects to 4 modes, JW-maps to 4 qubits, builds H_C, runs PSO seeding, and executes a $p = 1$QAOA optimization on a statevector simulator. **I will include the explicit matrices, PSO hyperparameters, and a QAOA circuit diagram.**

Risks: finite-size artifacts and parity constraints require careful mixer choice; NISQ noise may obscure Pfaffian/edge signatures—mitigate by symmetry-preserving ansatz and classical pretraining.

A runnable Python notebook (NumPy + SciPy + Qiskit/Stim) that: constructs the 3-site TB, forms BdG, projects to 4 modes, JW-maps to 4 qubits, builds $H\ C$, runs PSO seeding, and

executes a p = 1 QAOA optimization on a statevector simulator. include the explicit matrices, PSO hyperparameters, and a QAOA circuit diagram.

I'm sorry, I'm having trouble responding to requests right now. Let's try this again in a bit.

produce A runnable Python notebook (NumPy + SciPy + Qiskit/Stim) that: constructs the 3-site TB, forms BdG, projects to 4 modes, JW-maps to 4 qubits, builds $H\ C$, runs PSO seeding, and executes a p = 1 QAOA optimization on a statevector simulator. include the explicit matrices, PSO hyperparameters, and a QAOA circuit diagram.

%% [markdown]

Minimal TB → BdG → QAOA pipeline with PSO seeding

This runnable Jupyter-style notebook (jupytext `.py` format) constructs a minimal 3-site Kagome tight-binding (TB) model, forms a Bogoliubov–de Gennes (BdG) Hamiltonian, projects to a 4-mode low-energy subspace, maps the resulting fermionic many-body Hamiltonian to 4 qubits via Jordan–Wigner (by building the full Fock-space operator and converting to Pauli strings), and runs a depth-$p = 1$ QAOA ansatz whose angles are **seeded by a simple Particle Swarm Optimization (PSO)**. The notebook uses `numpy`, `scipy`, `qiskit`, and `matplotlib`. It is self-contained and uses explicit random seeds for reproducibility.

Notes

- This is a minimal, pedagogical pipeline intended to run on a classical statevector simulator (Qiskit Aer or Qiskit's `Statevector`).
- The cost evolution is implemented by exponentiating the many-body Hamiltonian matrix; the QAOA circuit uses those unitaries as `Operator` instructions so we can draw a compact circuit diagram.
- If Qiskit is not installed, the imports cell prints instructions to install it.

%%

%% [markdown]

1) Imports and environment checks

%%

Core imports

import numpy as np np.set_printoptions(precision=4, suppress=True) from scipy.linalg import eigh, expm from scipy.sparse.linalg import eigsh import matplotlib.pyplot as plt

Qiskit imports with fallback message

try: from qiskit import QuantumCircuit, Aer, transpile from qiskit.quantum_info import Operator, Statevector, SparsePauliOp from qiskit.quantum_info import Pauli qiskit_available = True except Exception as e: qiskit_available = False print("Qiskit not available. Install with: pip install qiskit") print("Proceeding will fail at Qiskit-dependent steps.")

Reproducibility

np.random.seed(42)

%%

%% [markdown]

2) Minimal 3-site Kagome tight-binding (TB) Hamiltonian

- Sites labeled 0,1,2

- Nearest-neighbor hopping t and chemical potential mu

- We choose a simple gauge / adjacency for a single triangle (Kagome unit)

%%

TB parameters

t = 1.0 mu = 0.0

adjacency for a triangle (0-1,1-2,2-0)

h_TB = np.array([[-mu, -t, -t], [-t, -mu, -t], [-t, -t, -mu]], dtype=float)

print("3x3 TB Hamiltonian h_TB:") print(h_TB)

%%

%% [markdown]

3) Build BdG Hamiltonian (6x6) with on-site s-wave pairing `Delta0`

- Nambu spinor ordering: (c0, c1, c2, c0^\u2020, c1^\u2020, c2^\u2020)

- BdG block structure:

[[h_TB - mu*I, Delta],

[Delta^, -*(h_TB - mu*I)^T]]

%%

Delta0 = 0.5 # on-site pairing amplitude (real) n_sites = 3

particle block

h_p = h_TB.copy()

pairing block (on-site s-wave)

Delta = np.eye(n_sites) * Delta0

assemble BdG

top = np.concatenate([h_p, Delta], axis=1) bottom = np.concatenate([Delta.conj(), -h_p.T], axis=1) H_BdG = 0.5 * np.concatenate([top, bottom], axis=0) # factor 1/2 conventional for Nambu print("\nBdG 6x6 matrix (Delta0=%.2f):" % Delta0) print(H_BdG)

diagonalize BdG (Hermitian)

eigvals, eigvecs = eigh(H_BdG) print("\nBdG eigenvalues:") print(np.round(eigvals, 6))

%%

%% [markdown]

4) Projection to a 4-mode low-energy subspace

- Select two lowest positive-energy BdG eigenmodes and their particle-hole partners

- Build projector onto those 4 eigenvectors and compute the projected BdG (4x4)

- Then extract effective single-particle (h_eff) and pairing (Delta_eff) blocks in Nambu ordering

%%

find indices of positive eigenvalues

pos_idx = np.where(eigvals > 1e-8)[0]

choose two smallest positive eigenvalues

if len(pos_idx) < 2: raise RuntimeError("Not enough positive BdG eigenvalues found for projection.") chosen_pos = pos_idx[:2]

find their particle-hole partners (negative eigenvalues with opposite magnitude)

For simplicity, pick the corresponding negative-indexed eigenvectors by symmetry:

choose the two smallest-magnitude negative eigenvalues

neg_idx = np.where(eigvals < -1e-8)[0] chosen_neg = neg_idx[:2]

chosen_indices = np.concatenate([chosen_pos, chosen_neg]) chosen_indices.sort() print("\nChosen BdG eigenvalue indices for projection:", chosen_indices) print("Chosen eigenvalues:", np.round(eigvals[chosen_indices], 6))

build projector matrix (6 x 4)

U_sub = eigvecs[:, chosen_indices] # columns are eigenvectors

projected BdG in subspace

H_proj = U_sub.conj().T @ H_BdG @ U_sub # 4x4 print("\nProjected BdG (4x4):") print(np.round(H_proj, 6))

split into Nambu blocks: first half = particles, second half = holes

Here we interpret the 4-mode subspace as Nambu-ordered: [q1,q2,q1^{dag,q2}dag]

To extract single-particle and pairing parts, we re-order if necessary.

For clarity, we will treat H_proj as an effective BdG in the chosen basis.

Print it and proceed to build many-body Hamiltonian from it.

H_eff_BdG = H_proj

%%

%% [markdown]

5) Build many-body fermionic operators (Fock basis) for up to 4 fermionic modes

- We'll construct annihilation/creation operators in the 2^4 Fock basis

- Then assemble the many-body Hamiltonian from the projected BdG blocks:

H_many = sum_{ij} h_ij c_i^\u2020 c_j + 1/2 sum_{ij} (Delta_ij c_i^\u2020 c_j^\u2020 + h.c.)

- For simplicity we extract effective single-particle and pairing matrices by splitting H_eff_BdG

into particle/hole quadrants assuming ordering [p1,p2,h1,h2] (approximate mapping)

%%

n_modes_sub = 2 # number of quasiparticle modes chosen (so 2 particle + 2 hole -> 4) n_qubits = 4 # map to 4 qubits (we will represent the many-body Fock space of 4 fermionic modes)

Build Fock-space creation/annihilation operators for 4 fermionic modes (2^4 x 2^4)

dim = 2 ** n_qubits

single-site operators

def kron_list(mats): res = mats[0] for m in mats[1:]: res = np.kron(res, m) return res

Pauli matrices for convenience

I = np.array([[1,0],[0,1]], dtype=complex) sigma_x = np.array([[0,1],[1,0]], dtype=complex) sigma_y = np.array([[0,-1j],[1j,0]], dtype=complex) sigma_z = np.array([[1,0],[0,-1]], dtype=complex) proj0 = np.array([[1,0],[0,0]], dtype=complex) proj1 = np.array([[0,0],[0,1]], dtype=complex) sigma_plus = np.array([[0,1],[0,0]], dtype=complex) sigma_minus = np.array([[0,0],[1,0]], dtype=complex)

Build fermionic annihilation operators using Jordan-Wigner strings

def fermionic_annihilation_op(mode, n_modes): """Return 2^n x 2^n matrix for annihilation operator a_mode (Jordan-Wigner)""" ops = [] for i in range(n_modes): if i < mode: ops.append(sigma_z) elif i == mode: ops.append(sigma_minus) else: ops.append(I) return kron_list(ops)

def fermionic_creation_op(mode, n_modes): return fermionic_annihilation_op(mode, n_modes).conj().T

Precompute creation/annihilation for 4 modes

a_ops = [fermionic_annihilation_op(i, n_qubits) for i in range(n_qubits)] adag_ops = [op.conj().T for op in a_ops]

For demonstration we will embed the 2-mode effective BdG into a 4-mode many-body model by placing

the effective single-particle and pairing blocks into the first two fermionic modes and leaving the

remaining two modes inert. This yields a 4-mode many-body Hamiltonian matrix we can convert to Pauli form.

For mapping, we need h_eff (2x2) and Delta_eff (2x2). We'll extract them heuristically from H_eff_BdG:

assume H_eff_BdG block structure [[h_eff, Delta_eff],[Delta_eff^*, -h_eff^T]]

half = H_eff_BdG.shape[0] // 2 h_eff = H_eff_BdG[:half, :half] Delta_eff = H_eff_BdG[:half, half:] print("\nExtracted h_eff (2x2):") print(np.round(h_eff, 6)) print("\nExtracted Delta_eff (2x2):") print(np.round(Delta_eff, 6))

Build many-body Hamiltonian (2^4 x 2^4)

H_many = np.zeros((dim, dim), dtype=complex)

embed h_eff and Delta_eff into modes 0 and 1 (leave modes 2 and 3 as spectators)

modes_active = [0,1] for i_local, i_mode in enumerate(modes_active): for j_local, j_mode in enumerate(modes_active): # single-particle term t_{ij} c_i^\u2020 c_j H_many += h_eff[i_local, j_local] * adag_ops[i_mode] @ a_ops[j_mode] # pairing term Delta_{ij} c_i^\u2020 c_j^\u2020 + h.c. H_many += 0.5 * Delta_eff[i_local, j_local] * adag_ops[i_mode] @ adag_ops[j_mode] H_many += 0.5 * np.conj(Delta_eff[i_local, j_local]) * a_ops[j_mode] @ a_ops[i_mode]

Make Hermitian

H_many = 0.5 * (H_many + H_many.conj().T) print("\nMany-body Hamiltonian matrix H_many (shape %s):" % (H_many.shape,)) print(np.round(H_many, 6))

%%

%% [markdown]

6) Convert many-body Hamiltonian to Pauli decomposition (SparsePauliOp)

- Use Qiskit's `SparsePauliOp.from_operator` if available

- Print Pauli terms and coefficients (sparse list)

%%

if not qiskit_available: raise RuntimeError("Qiskit is required for Pauli decomposition and circuit construction.")

Convert to SparsePauliOp

H_op = Operator(H_many) sparse_pauli = SparsePauliOp.from_operator(H_op)

compress and list terms

sparse_pauli = sparse_pauli.reduce() print("\nNumber of Pauli terms in H_C:", len(sparse_pauli.paulis))

Print first 20 terms (Pauli string and coefficient)

print("\nSample Pauli terms (up to 20):") for i in range(min(20, len(sparse_pauli.paulis))): p = sparse_pauli.paulis[i] coeff = sparse_pauli.coeffs[i] print(f"{p.to_label()} : {coeff:.6f}")

We'll treat H_C as the cost Hamiltonian

H_C = sparse_pauli

For expectation evaluation we will use the dense matrix H_many directly.

%%

%% [markdown]

7) QAOA ansatz (p=1) builder

- We prepare \|+\>^{\otimes n} initial state

- Apply cost unitary U_C = exp(-i * gamma * H_many)

- Apply mixer unitary U_M = exp(-i * beta * H_M)

- Two mixer choices:

* 'X' : H_M = sum_j X_j (standard)

* 'parity' : parity-preserving mixer implemented as pairwise XX rotations on (0,1) and (2,3)

%%

from qiskit.circuit.library import EfficientSU2

def build_qaoa_unitaries(gamma, beta, mixer='X'): """Return unitary matrices U_C and U_M for given angles.""" U_C = expm(-1j * gamma * H_many) # cost unitary (many-body) # build mixer Hamiltonian matrix H_M_mat = np.zeros_like(H_many, dtype=complex) if mixer == 'X': # sum of single-qubit X operators for q in range(n_qubits): # build X on qubit q ops = [] for i in range(n_qubits): ops.append(sigma_x if i == q else I) H_M_mat += kron_list(ops) elif mixer == 'parity': # parity-preserving: pairwise XX on (0,1) and (2,3) pairs = [(0,1), (2,3)] for (a,b) in pairs: ops = [] for i in range(n_qubits): if i == a or i == b: ops.append(sigma_x) else: ops.append(I) # but XX is sigma_x \otimes sigma_x on the pair; we need to ensure it's only on those two qubits # Construct XX for the pair explicitly: # build operator = I... X_a ... X_b ...I H_pair = kron_list([sigma_x if i==a or i==b else I for i in range(n_qubits)]) H_M_mat += H_pair else: raise ValueError("Unknown mixer")

U_M = expm(-1j * beta * H_M_mat) return U_C, U_M, H_M_mat

QAOA statevector from angles

def qaoa_statevector(gamma, beta, mixer='X'): # initial |+> state plus = (1/np.sqrt(2)) * np.array([1,1], dtype=complex) psi0 = plus for _ in range(n_qubits-1): psi0 = np.kron(psi0, plus) U_C, U_M, _ = build_qaoa_unitaries(gamma, beta, mixer=mixer) # apply U_C then U_M (p=1): |psi> = U_M U_C |+> psi = U_M @ (U_C @ psi0) return psi

Build a Qiskit circuit that uses the unitaries as instructions (for drawing)

def qaoa_circuit_from_unitaries(gamma, beta, mixer='X'): U_C, U_M, _ = build_qaoa_unitaries(gamma, beta, mixer=mixer) qc = QuantumCircuit(n_qubits) # prepare |+>^n for q in range(n_qubits): qc.h(q) # append cost unitary as instruction qc.append(Operator(U_C).to_instruction(), qc.qubits) # append mixer qc.append(Operator(U_M).to_instruction(), qc.qubits) return qc

Example circuit diagram

gamma_test = 1.0 beta_test = 0.5 qc_example = qaoa_circuit_from_unitaries(gamma_test, beta_test, mixer='X') print("\nQAOA circuit (p=1) summary:") print(qc_example.draw(output='text'))

Render a matplotlib diagram

fig = qc_example.draw(output='mpl', scale=0.8) plt.show()

%%

%% [markdown]

8) PSO seeding for (gamma, beta)

- Simple PSO implementation in pure Python

- Search bounds: gamma in [0, 2*pi], beta in [0, pi]

- swarm_size=12, inertia=0.7, cognitive=1.5, social=1.5, iterations=40

- Cost function: expectation value E(gamma,beta) = <psi|H_many|psi>

%%

import math from copy import deepcopy

PSO hyperparameters

swarm_size = 12 inertia = 0.7 c1 = 1.5 c2 = 1.5 iterations = 40 gamma_bounds = (0.0, 2*np.pi) beta_bounds = (0.0, np.pi)

initialize swarm

np.random.seed(42) swarm_pos = np.column_stack([np.random.uniform(*gamma_bounds, size=swarm_size), np.random.uniform(*beta_bounds, size=swarm_size)]) swarm_vel = np.zeros_like(swarm_pos) pbest_pos = swarm_pos.copy() pbest_val = np.full(swarm_size, np.inf) gbest_pos = None gbest_val = np.inf

evaluate function

def energy_from_angles(gamma, beta, mixer='X'): psi = qaoa_statevector(gamma, beta, mixer=mixer) E = np.real(np.vdot(psi, H_many @ psi)) return E

PSO loop

history = [] for it in range(iterations): for i in range(swarm_size): g, b = swarm_pos[i] val = energy_from_angles(g, b, mixer='X') if val < pbest_val[i]: pbest_val[i] = val pbest_pos[i] = swarm_pos[i].copy() if val < gbest_val: gbest_val = val gbest_pos = swarm_pos[i].copy() # record history.append((swarm_pos.copy(), pbest_pos.copy(), pbest_val.copy(), gbest_pos.copy(), gbest_val)) # update velocities and positions for i in range(swarm_size): r1 = np.random.rand(2) r2 = np.random.rand(2) swarm_vel[i] = inertia * swarm_vel[i] + c1 * r1 * (pbest_pos[i] - swarm_pos[i]) + c2 * r2 * (gbest_pos - swarm_pos[i]) swarm_pos[i] = swarm_pos[i] + swarm_vel[i] # clamp to bounds swarm_pos[i,0] = np.clip(swarm_pos[i,0], *gamma_bounds) swarm_pos[i,1] = np.clip(swarm_pos[i,1], *beta_bounds) if (it+1) % 10 == 0 or it==0: print(f"PSO iter {it+1}/{iterations}, gbest_val = {gbest_val:.6f}")

print("\nPSO best seed (gamma,beta) = ", gbest_pos, "energy =", gbest_val)

plot swarm trajectories (gamma vs beta)

plt.figure(figsize=(6,5)) for i in range(swarm_size): traj = np.array([h[0][i] for h in history]) plt.plot(traj[:,0], traj[:,1], '-o', markersize=3, alpha=0.6) plt.scatter([h[3][0] for h in history], [h[3][1] for h in history], c='red', label='gbest over time', s=20) plt.xlabel('gamma') plt.ylabel('beta') plt.title('PSO particle trajectories (gamma vs beta)') plt.grid(True) plt.show()

%%

%% [markdown]

9) Hybrid local optimization (refine best PSO seed)

- Use `scipy.optimize.minimize` with Nelder-Mead starting from PSO best

- Objective: energy_from_angles

%%

from scipy.optimize import minimize

x0 = gbest_pos.copy() def obj(x): g, b = float(x[0]), float(x[1]) return energy_from_angles(g, b, mixer='X')

res = minimize(obj, x0, method='Nelder-Mead', options={'maxiter':200, 'xatol':1e-6, 'fatol':1e-6}) print("\nLocal optimization result:") print(res) gamma_opt, beta_opt = res.x E_opt = res.fun print(f"\nOptimized angles: gamma={gamma_opt:.6f}, beta={beta_opt:.6f}, energy={E_opt:.6f}") print(f'PSO seed energy: {gbest_val:.6f}")

%%

%% [markdown]

10) Results: energy landscape and overlay of PSO + optimized point

- Compute coarse grid of energies and plot contour

- Overlay PSO particle final positions and the optimized point

%%

coarse grid

g_grid = np.linspace(0, 2*np.pi, 60) b_grid = np.linspace(0, np.pi, 40) G, B = np.meshgrid(g_grid, b_grid) E_grid = np.zeros_like(G) for i in range(G.shape[0]): for j in range(G.shape[1]): E_grid[i,j] = energy_from_angles(G[i,j], B[i,j], mixer='X')

plt.figure(figsize=(8,5)) cs = plt.contourf(G, B, E_grid, levels=30, cmap='viridis') plt.colorbar(cs, label='Energy')

overlay final swarm positions

final_swarm = history[-1][0] plt.scatter(final_swarm[:,0], final_swarm[:,1], c='white', edgecolor='k', label='final swarm', s=40) plt.scatter([gamma_opt], [beta_opt], c='red', label='optimized', s=80) plt.xlabel('gamma') plt.ylabel('beta') plt.title('Energy landscape with PSO final swarm and optimized point') plt.legend() plt.show()

Compute final state fidelity with projector onto chosen BdG subspace

psi_opt = qaoa_statevector(gamma_opt, beta_opt, mixer='X')

projector onto the low-energy subspace in Fock basis: build projector from U_sub eigenvectors mapped to many-body basis

For simplicity we compute overlap of psi_opt with the subspace spanned by the many-body eigenvectors of H_many with lowest energies

vals_many, vecs_many = eigh(H_many)

take k lowest eigenvectors (k = 4)

k = 4 subspace_vecs = vecs_many[:, :k]

compute fidelity as sum of squared overlaps with subspace

overlaps = np.abs(subspace_vecs.conj().T @ psi_opt)**2 fidelity = overlaps.sum() print(f"\nFidelity of optimized QAOA state with low-energy many-body subspace (k={k}): {fidelity:.6f}")

Print expectation values of a few Pauli strings (first 8)

print("\nExpectation values of first Pauli terms in H_C for optimized state:") for i in range(min(8, len(sparse_pauli.paulis))): pauli = sparse_pauli.paulis[i] coeff = sparse_pauli.coeffs[i] # build operator matrix for this Pauli op_mat = Pauli(pauli.to_label()).to_matrix() # Pauli.to_matrix returns 2^n x 2^n if label length equals n_qubits expval = np.vdot(psi_opt, op_mat @ psi_opt) print(f"{pauli.to_label():>8} coeff={coeff:.6f} <P>={np.real(expval):.6f}")

%%

%% [markdown]

11) Notes and limitations

- This notebook demonstrates a compact, end-to-end pipeline for a minimal TB→BdG→QAOA workflow.

- Limitations

- The projection step is heuristic: selecting BdG eigenvectors and embedding into a 4-mode many-body model is simplified for pedagogy.

- The cost unitary is implemented by dense matrix exponentiation; this is not scalable beyond small Hilbert spaces.

- On real NISQ hardware, one must decompose the cost and mixer unitaries into native gates and respect connectivity and noise.

- Extensions

- Increase p and implement parameter concentration strategies.

- Replace dense exponentiation with Trotterized Pauli-evolution circuits using the Pauli decomposition H_C.

- Use parity-preserving mixers and symmetry-aware encodings for Majorana/particle-hole sectors.

- Replace PSO with more advanced global optimizers or hybrid strategies (PSO seed + gradient-based fine-tuning).

End of notebook.